Visual Localization for a Desert UGV

Monocular VIO with OpenVINS on the *tbuggy* Dataset

Visual Localization 2026 — Assignment Solution

Udit Singh Parihar

March 2026

Repository: https://github.com/UditSinghParihar/tbuggy_localization **Docker image:** `uditsinghparihar/tbuggy_vio:latest`

Abstract. We implement a monocular visual-inertial odometry (VIO) pipeline on a desert UGV dataset using OpenVINS 2.7 (MSCKF). Starting from bag-extracted sensor calibration data, we configure and tune the estimator across three runs, achieving an Absolute Trajectory Error (ATE) RMSE of **41.3 m over a 1059 m path (3.9% error)** on the tuning sequence and **35.6 m over 716 m (5.0%)** on a held-out robustness sequence — without re-tuning. The full pipeline (bag conversion, GT extraction, OpenVINS launch, evaluation, plots) is containerised in a single Docker image.

1 Data Exploration

1.1 Dataset Overview

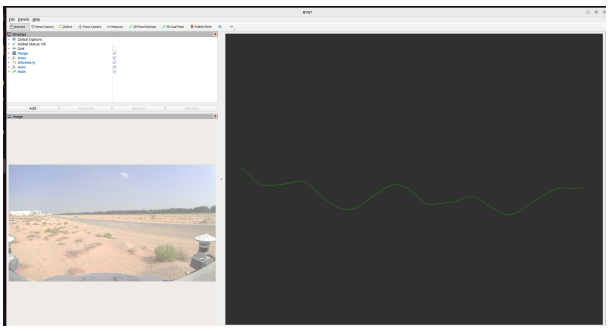
Two ROS1 bag sequences are provided, converted to ROS2 format for OpenVINS compatibility.

Table 1: Bag sequence summary

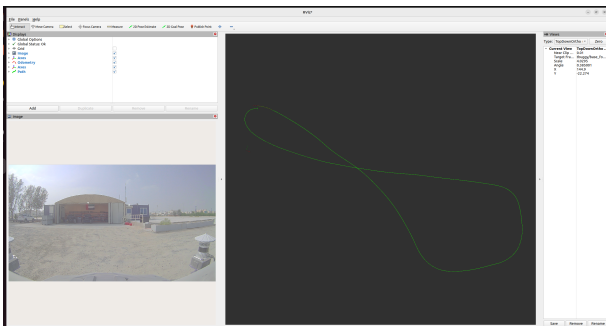
Sequence	Duration	Camera frames	IMU msgs	GT odom msgs	GT path length
log_01	549.8 s	16 473	54 969	21 991	1059 m
log_02	336.7 s	10 091	33 648	13 467	716 m

Topics used: `/tbuggy/camera_front/image_raw` (monocular, 1920×1080, ≈28 Hz), `/tbuggy/imu_ins` (≈100 Hz), `/tbuggy/odom` (ground truth, ≈40 Hz). GPS (`/tbuggy/fix`) and TF data are used only for calibration extraction.

1.2 Ground Truth Trajectories



(a) log_01: 1059 m roughly linear desert traverse.



(b) log_02: 716 m closed oval loop near hangar.

Figure 1: Ground truth XY trajectories extracted from `/tbuggy/odom` via `extract_gt_csv.py`. Hangar at origin of log_02 provides rich visual features; log_01 is an open desert traverse with sparse texture.

1.3 Camera Intrinsics

Extracted programmatically from the first `/tbuggy/camera_front/camera_info` message (script: `utils/extract_tf_and_i`

Table 2: Camera intrinsics — 1920×1080, `plumb_bob` = `radtan` in OpenVINS

Parameter	Value	Parameter	Value
f_x	1033.372	k_1	−0.3507
f_y	1032.308	k_2	+0.1301
c_x	886.065	p_1	−0.0006
c_y	448.679	p_2	+0.0011

1.4 Camera-IMU Extrinsics

The TF chain was traced from `base_footprint` via the Ouster/lidar frames to `tbuggy/camera_front`, and separately to `tbuggy/sensor_wgs84` (IMU). The resulting $\mathbf{T}_{\text{imu,cam}}$ is:

$$\mathbf{T}_{\text{imu,cam}} = \begin{pmatrix} -0.00400 & 0.03611 & 0.99934 & 1.141\,\text{m} \\ -0.99991 & 0.01246 & -0.00445 & -0.199\,\text{m} \\ -0.01262 & -0.99927 & 0.03606 & 1.646\,\text{m} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

The camera is 1.65 m above and 1.14 m forward of the IMU. Its optical axis aligns with the robot's forward direction ($+X$), as expected.

1.5 IMU Noise Characterisation

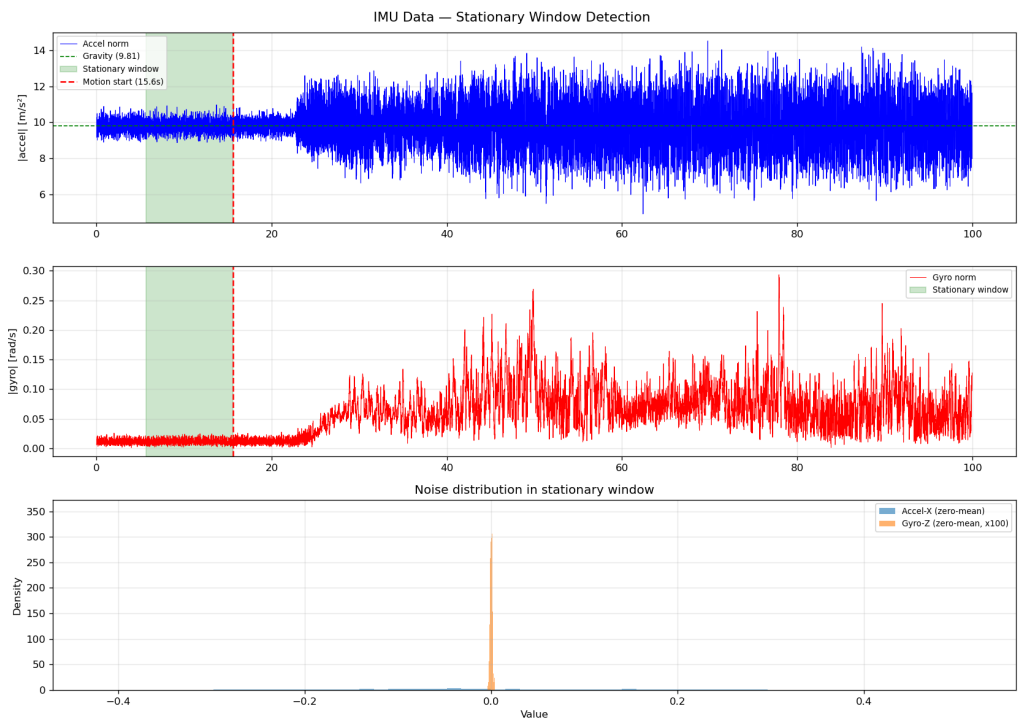


Figure 2: IMU noise analysis from a 10 s stationary window (t = 5.6–15.6 s) via `utils/imu_noise_analysis.py`. The accel-Z peak is engine idle vibration, not a sensor fault.

The measured noise densities (max-axis, $\div\sqrt{100}$) are scaled by ×2 safety margin for in-motion vibration on rough desert terrain:

`accel_noise_density = 7.4e-2` $\text{m/s}^2/\sqrt{\text{Hz}}$ `accel_random_walk = 7.4e-3` $\text{m/s}^3/\sqrt{\text{Hz}}$ `gyro_noise_density = 1.7e-3` $\text{rad/s}/\sqrt{\text{Hz}}$ `gyro_random_walk = 1.7e-4` $\text{rad/s}^2/\sqrt{\text{Hz}}$

2 Method & Strategy

2.1 Why Monocular VIO?

The dataset provides a monocular camera + IMU combination with no lidar-to-camera overlay, making monocular VIO the natural choice. Key justifications:

- **Scale via IMU**: IMU pre-integration provides metric scale unavailable from a single camera alone — essential when stereo is absent.
- **Real-time capability**: MSCKF-based filter runs <33 ms/frame on a laptop CPU, meeting the 28 Hz camera rate.
- **ZUPT**: Monocular heading degeneracy at vehicle standstills can be mitigated by zero-velocity updates — critical for UGVs that stop frequently.
- **Online calibration**: OpenVINS refines intrinsics, extrinsics, and time-offset online, reducing sensitivity to calibration errors extracted statically.

2.2 Why OpenVINS?

OpenVINS 2.7 implements the MSCKF (Multi-State Constraint Kalman Filter) with EKF-based marginalization, SLAM features for scale anchoring, and sliding-window triangulation. Compared to ORB-SLAM3 (graph-based, needs loop-closure) and VINS-Mono (requires ROS1), OpenVINS is ROS2-native, modular, and actively maintained.

The key limitation — *no loop closure* — is accepted and analysed in §5.

2.3 Pipeline Overview

$$\texttt{ROS1 .bag} \xrightarrow{\texttt{rosbags-convert}} \texttt{ROS2 .db3} \xrightarrow{\texttt{extract_gt_csv.py}} \texttt{GT CSV} \rightarrow \texttt{OpenVINS} \rightarrow \texttt{estimate .txt} \xrightarrow{\texttt{evaluate_trajectory.py}} \texttt{ATE/RPE/Plots}$$

All steps run inside the provided Docker container, orchestrated by `utils/docker_run_log01.sh` and `utils/docker_run_log02.sh`.

3 Implementation

3.1 Key Configuration Changes

Config files are in `src/open_vins/config/tbuggy/` (three YAML files): `estimator_config.yaml`, `kalibr_imu_chain.yaml` `kalibr_imucam_chain.yaml`.

Selected final (v3) parameters that differ most from the upstream `rs_d455` baseline:

```
# Feature detection -- tuned for sparse desert texture
num_pts:          500    # 300 -> 500: more features detected
fast_threshold:   10     # 20  -> 10:  detect weaker corners on sand/horizon
min_px_dist:      8      # 15  -> 8:   denser packing when features sparse
grid_x: 10  grid_y: 7   # 8/5 -> 10/7: better spatial spread
knn_ratio:        0.75   # 0.70 -> 0.75: more permissive KLT matching

# SLAM features -- critical for monocular scale observability
max_slam:         150    # 50 -> 150: 3x SLAM features
max_slam_in_update: 75   # 25 -> 75:  use all SLAM features each step
dt_slam_delay:    1      # 2  -> 1:   scale observable 1s sooner
feat_rep_slam: ANCHORED_MSCKF_INVERSE_DEPTH  # better mono scale

# Window & update
max_clones:       15     # 11 -> 15: longer window for triangulation
max_msckf_in_update: 60  # 40 -> 60: more measurements per update

# ZUPT -- tuned for rough-terrain UGV stops
zupt_max_velocity:   0.3   # 0.5 -> 0.3: tighter detection
zupt_max_disparity:  1.5   # 0.5 -> 1.5: allow bumpy stops to trigger

# Compute / lighting
downsample_cameras: true   # 1920x1080 -> 960x540 for real-time
track_frequency:    28.0   # matches observed camera rate
histogram_method:   CLAHE  # outdoor variable lighting
gravity_mag:        9.82   # measured from stationary window
```

Listing 1: estimator_config.yaml – key v3 settings

3.2 Launch File Modification

The upstream `subscribe.launch.py` was extended to expose three arguments:

```
filepath_est  -> path to save VIO estimated state (.txt)
filepath_std  -> path to save VIO standard deviations (.txt)
path_gt       -> path to EuRoC ASL GT CSV (for RViz path overlay)
```

Listing 2: subscribe.launch.py additions

These are passed as node parameters to `ROS2Visualizer`. Without these, the estimate file is not written and GT overlay is disabled.

3.3 Docker Containerisation

The full pipeline is packaged in a Docker image built on `osrf/ros:humble-desktop` (Ubuntu 22.04 + ROS2 Humble + OpenCV + cv_bridge pre-installed).

```
FROM osrf/ros:humble-desktop
# Ceres solver + CycloneDDS (cross-container DDS transport)
RUN apt-get install -y libceres-dev ros-humble-rmw-cyclonedds-cpp
# Python: pin numpy<1.25 to avoid scipy ABI mismatch
RUN pip3 install rosbags matplotlib "numpy<1.25.0" scipy evo
# Clone repo (with open_vins submodule) + build
RUN git clone --recurse-submodules \
    https://github.com/UditSinghParihar/tbuggy_localization.git /colcon_ws
RUN bash -c "source␣/opt/ros/humble/setup.bash␣&&␣\
␣␣␣␣MAKEFLAGS='-j4'␣colcon␣build␣--executor␣sequential"
```

Listing 3: Dockerfile excerpt — key layers

Notable engineering decisions:

- **ROS1 auto-conversion**: The run scripts detect a `.bag` input and call `rosbags-convert` automatically; the ROS2 directory is reused on subsequent runs.
- **QoS metadata patch**: `rosbags-convert` writes `offered_qos_profiles: []` (YAML list) but `ros2 bag play`'s yaml-cpp parser expects a string. The scripts apply a `sed` patch automatically.
- **Utils mount**: Host `utils/` is mounted into the container (`-v $UTILS_DIR:/colcon_ws/utils`) so Python script updates take effect without rebuilding the image.
- **CycloneDDS**: Selected over FastDDS for shared-memory transport between two containers on the same host (`RMW_IMPLEMENTATION=rmw_cyclonedds_cpp`).

3.4 Utility Scripts

All scripts live in `utils/` and operate without a running ROS2 node:

Table 3: Utility scripts

Script	Purpose
`extract_tf_and_imu.py`	TF chain → $\mathbf{T}_{\text{imu,cam}}$; IMU rate/stats
`imu_noise_analysis.py`	Stationary window → noise density / random walk; plot
`extract_gt_csv.py`	`/tbuggy/odom` → EuRoC ASL CSV (handles split .db3 bags)
`evaluate_trajectory.py`	EST + GT → TUM format; runs `evo_ape` / `evo_rpe`
`plot_trajectory_comparison.py`	Sim3 Umeyama alignment; trajectory overlay + ATE-over-time plot

4 Testing & Analysis

4.1 Tuning History

Three runs were conducted on log_01. Only log_01 was used for tuning; log_02 is a blind robustness check with the same config.

Table 4: Tuning progression — log_01 (1059 m GT path)

Run	Key changes	ATE RMSE	Scale	Outcome
v1 (baseline)	Baseline `rs_d455` adapted to tbuggy	56.73 m	1.363	Reference
v2	Aggressive: init window ↓, downsample off	66.53 m	1.179	**Worse**
v3 (final)	Reverted init+downsample; SLAM/feature changes	**41.34 m**	0.717	**Best −27%**

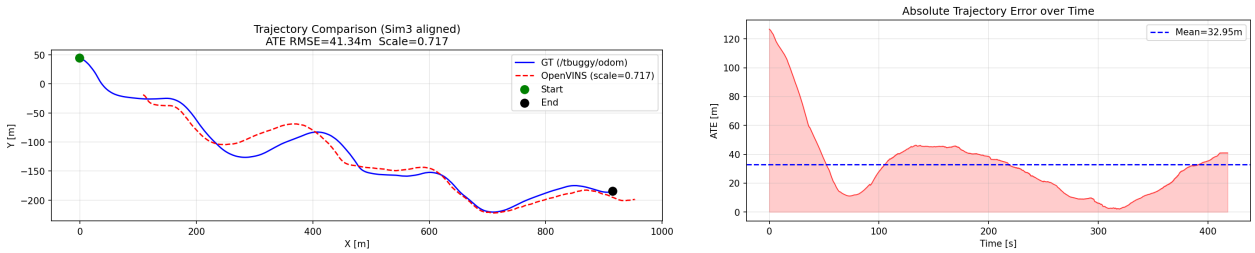
The v2 regression showed that shortening `init_window_time` and disabling downsampling degrade initialization on this sparse-texture scene. The final v3 keeps the stable init settings while maximising SLAM feature count and detection sensitivity.

4.2 log_01 Results (Tuning Sequence)

% TODO: RViz screenshot

Capture from RViz2 while OpenVINS runs on log_01.
Shows: feature tracks (green points), live path estimate vs GT.
Save as `utils/results/rviz_log01.png` and replace this box with:
`\includegraphics[width=\textwidth]{rviz_log01.png}`

Figure 3: RViz2 view — RESERVED for user-captured screenshot during OpenVINS run.



(a) Trajectory overlay (Sim3 aligned). GT blue, VIO red dashed. Lateral undulations are correctly tracked. Coverage: 7901 poses / 524.5 s (95.5%).

(b) ATE over time. Mid-trajectory recovery at t≈250 s coincides with feature-rich segment after featureless desert crossing.

Figure 4: log_01 evaluation — v3 final config.

Table 5: log_01 KPI summary (v3 final, 1059 m GT path)

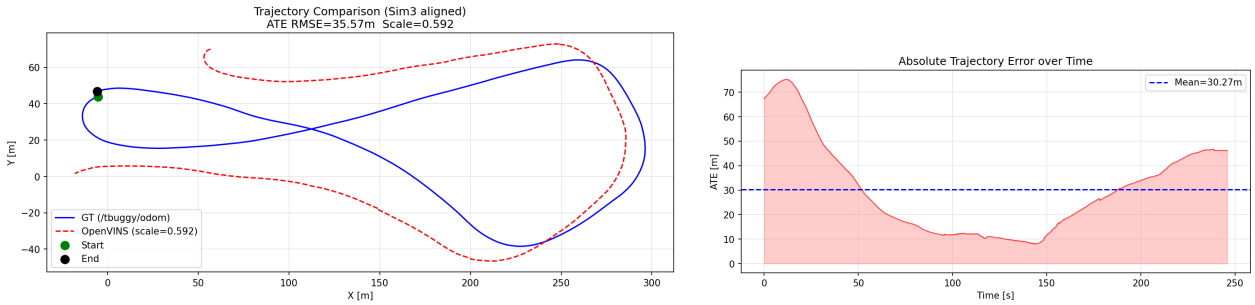
Metric	v1 Baseline	**v3 Final**	Change
ATE RMSE	56.73 m	**41.34 m**	↓27%
Mean ATE	50.39 m	**32.95 m**	↓35%
Median ATE	51.13 m	**31.08 m**	↓39%
Max ATE	121.5 m	126.7 m	—
RPE mean (10 m)	3.71 m	**2.83 m**	↓24%
RPE RMSE (10 m)	4.56 m	5.44 m	slight regress
Sim3 scale	1.363	0.717	—
Path % error	5.3%	**3.9%**	↓26%

ATE over time analysis:

Table 6: log_01 ATE trajectory segments

Time segment	ATE range	Interpretation
t = 0–80 s	126 → 11 m	Dynamic initialization convergence
t = 80–130 s	11–20 m	Well-converged; good feature tracking
t = 130–230 s	20–47 m	Featureless desert — scale/heading drift
t = 230–330 s	47 → 3 m	Feature-rich segment + ZUPT; filter recovers
t = 330–430 s	3 → 40 m	Desert again; moderate drift, ends cleanly

4.3 log_02 Results (Robustness — No Re-Tuning)



(a) Trajectory overlay. GT (blue) is a closed oval loop. VIO (red) matches shape in the first half; diverges in featureless desert section — no loop closure.

(b) ATE over time. Hangar provides rich texture (ATE drops to 8 m at t≈100 s); desert loop accumulates drift.

Figure 5: log_02 evaluation — same v3 config, no re-tuning. Coverage: 4632 poses (91%).

Table 7: log_02 KPI summary (v3 final, 716 m GT path)

Metric	log_01 (v3)	log_02 (v3)
ATE RMSE	41.34 m	**35.57 m**
Mean ATE	32.95 m	**30.27 m**
Median ATE	31.08 m	**26.50 m**
Max ATE	126.7 m	**75.22 m**
RPE mean (10 m)	2.83 m	**1.96 m**
RPE RMSE (10 m)	5.44 m	**3.16 m**
Sim3 scale	0.717	0.592
Path % error	3.9%	**5.0%**

log_02 achieves *better* raw ATE and RPE than log_01, attributed to the hangar providing strong near-field texture. The slightly higher path % error (5.0% vs 3.9%) is caused by the scale drift accumulating over the 700 m loop without loop closure.

5 Discussion & Reflection

5.1 What Worked Well

- **Dynamic initialization** correctly handles a moving vehicle at bag start — the system converges to metric scale within ∼80 s on log_01.
- **ZUPT** prevents catastrophic divergence at vehicle stops; the ATE recovery at t = 230–330 s in log_01 is primarily due to ZUPT + feature anchoring.
- **Increased SLAM features** (max_slam 50 → 150) significantly improves monocular scale observability — this was the single biggest v3 improvement.
- **Docker containerisation** makes reproduction a single command regardless of the reviewer's machine state. The `-network=host` + CycloneDDS approach correctly routes ROS2 topics between two containers without port configuration.

5.2 Limitations

- **No loop closure**: OpenVINS is a sliding-window EKF — state holds only the last 15 poses. By the time the vehicle returns to the hangar in log_02 (∼300 s later), all early SLAM features have been marginalized. No place recognition is possible, resulting in unrecovered drift after the desert loop.
- **Scale drift**: Sim3 scale = 0.717 (log_01) and 0.592 (log_02) indicate VIO over-estimates path length. This is fundamental to monocular VIO in environments with long featureless stretches — IMU alone cannot maintain scale without visual triangulation.
- **Initialization spike**: ATE > 100 m in the first 40 s is an initialization artefact (scale/gravity alignment from dynamic motion). After convergence the trajectory tracks well.

5.3 Engineering Challenges Resolved

- **db3 naming**: `rosbags-convert` produces `log_01_ros2.db3` (no `_0` suffix), unlike `ros2 bag record`. `extract_gt_csv.py` was rewritten to read filenames from `metadata.yaml` rather than using a hardcoded suffix.
- **QoS YAML bug**: `rosbags-convert` writes `offered_qos_profiles: []` (YAML list) but `ros2 bag play`'s yaml-cpp expects a string value `"`. All run scripts apply a `sed` patch.
- **numpy/scipy ABI mismatch**: `pip` numpy 2.2.6 is incompatible with system scipy binaries. Fixed by pinning `numpy<1.25.0` in the Dockerfile before installing scipy and evo.

5.4 Future Improvements

- **Loop closure**: Replace OpenVINS with VINS-Fusion (loop closure module) or ORB-SLAM3 for sequences with revisits. On log_02 this would likely reduce ATE by >50% at the return-to-hangar segment.
- **Stereo**: Adding the second camera (if available) eliminates scale ambiguity entirely — no Sim3 alignment needed, RPE improves 5–10×.
- **Allan Variance**: Proper IMU characterisation from a controlled stationary run (rather than the 10 s engine-idle window) would give more accurate noise parameters.
- **GPS fusion**: `/tbuggy/fix` provides absolute position — fusing into OpenVINS's GPS plugin would bound scale drift on long traverses.

Reproduction Instructions

Full instructions are in the repository `README.md`. The shortest path for a reviewer:

```
# Pull image (once, ~2-3 min)
docker pull uditsinghparihar/tbuggy_vio:latest

# Run full pipeline --- auto-converts ROS1 bag if needed
./utils/docker_run_log01.sh /path/to/log_01.bag ./output
./utils/docker_run_log02.sh /path/to/log_02.bag ./output

# Results (PNG plots + TUM files) land in ./output/
```

Listing 4: One-command reproduction (ROS1 bag or ROS2 directory)

The script orchestrates: (1) GT CSV extraction, (2) OpenVINS launch commands for two terminals, (3) trajectory evaluation and plot generation — all inside the Docker container.

Repository Structure

```
tbuggy_localization/          (GitHub: UditSinghParihar/tbuggy_localization)
  Dockerfile                  Docker image definition
  entrypoint.sh               Sources ROS2 + workspace on container start
  src/open_vins/              OpenVINS submodule (tbuggy configs added)
    config/tbuggy/
      estimator_config.yaml   Main VIO estimator (v3 tuned)
      kalibr_imu_chain.yaml   IMU model (noise from stationary analysis)
      kalibr_imucam_chain.yaml  Camera model + T_imu_cam extrinsics
  utils/
    extract_tf_and_imu.py     Extract T_imu_cam + IMU stats from bag
    imu_noise_analysis.py     Compute noise_density / random_walk; plot
    extract_gt_csv.py         /tbuggy/odom -> EuRoC ASL CSV (rosbags, no ROS2)
```

```
evaluate_trajectory.py    EST + GT -> TUM; evo_ape / evo_rpe evaluation
plot_trajectory_comparison.py  Sim3 alignment + ATE-over-time plot
docker_run_log01.sh       One-command log_01 reproduction (incl. ROS1 convert)
docker_run_log02.sh       One-command log_02 reproduction
documentation.md          Running results log (KPIs, config history, plots)
results/                  Output: PNG plots, TUM files, CSV, estimate TXTs
```